University Francisco José de Caldas

Department of Computer Engineering

Empire's Taste: A Medallion Data Pipeline for Gastronomic Trend Analysis and Sentiment Intelligence in NYC

Jairo Arturo Barrera Mosquera
Maria del Pilar Pradilla Cely

David Santiago Aldana Gonzalez

Supervisor: Eng. Carlos Andrés Sierra, M.Sc.

A report submitted in partial fulfilment of the requirements of
the University of Reading for the degree of
Master of Science in *Data Science and Advanced Computing*

June 4, 2026

Declaration

We, Jairo Arturo Barrera Mosquera, Maria del Pilar Pradilla Cely, and David Santiago Aldana Gonzalez, of the Department of Computer Engineering, University Francisco José de Caldas, confirm that this is our own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

We give consent to a copy of our report being shared with future students as an exemplar.

We give consent for our work to be made available more widely to members of UoR and the public with interest in teaching, learning, and research.

Jairo Arturo Barrera Mosquera
Maria del Pilar Pradilla Cely
David Santiago Aldana Gonzalez

June 4, 2026

Abstract

This report presents Empire's Taste, a course project that integrates the outputs of four workshops, the final paper, and the final poster into a complete technical report. The purpose of the project is to support data-driven event menu planning in New York City by combining structured recipe metadata with unstructured culinary media and diner review signals. The methodology follows a Medallion Architecture, moving data from raw Bronze JSON ingestion to cleaned Silver Parquet datasets and curated Gold PySpark aggregations, all orchestrated with Apache Airflow in a Docker Compose environment. Natural Language Processing methods, including TF-IDF keyword extraction, spaCy named entity recognition, and VADER sentiment scoring, transform Eater NY article summaries and OpenTable diner reviews into trend and sentiment features that can be linked to Spoonacular recipe recommendations. OpenTable was added in Workshop 4 because the original Eater NY-only sentiment view was overly positive and did not represent direct consumer satisfaction. Results include validated ingestion outputs, documented data quality metrics, governance KPIs, storytelling datasets, a combined sentiment distribution, and two Plotly Dash dashboards for technical monitoring and functional decision-making. The project concludes that an end-to-end data engineering pipeline can reduce intuition-based menu planning by giving catering managers and event planners auditable, trend-aware, diet-aware, and review-aware gastronomic insights.

Keywords: Data Engineering, Medallion Architecture, Apache Airflow, Sentiment Analysis, Natural Language Processing, Event Planning, Plotly Dash.

Total word count:7322
GitHub: https://github.com/Salda1308/Food-Gastronomy-Trends-Sentiment

Contents

List of Figures

List of Tables

List of Abbreviations

ABSA	Aspect-Based Sentiment Analysis
API	Application Programming Interface
CARS	Context-Aware Recommender Systems
DAG	Directed Acyclic Graph (Airflow)
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
NER	Named Entity Recognition
NLP	Natural Language Processing
RSS	Really Simple Syndication
SMPCS	School of Mathematical, Physical and Computational Sciences
TF-IDF	Term Frequency-Inverse Document Frequency
UoR	University of Reading
VADER	Valence Aware Dictionary and Sentiment Reasoner
XML	Extensible Markup Language

Chapter 1

Introduction

1.1 Background

Menu design for events in New York City is a high-impact decision in the hospitality industry. Hotels, restaurants, and catering teams must propose dishes that satisfy client expectations, respond to dietary constraints, and remain aligned with fast-changing culinary trends. The final poster and paper frame this challenge as a gap between abundant food-related data and the practical decisions made by catering managers: culinary news, restaurant openings, reviews, recipe metadata, and ingredient information are available, but they are rarely integrated into a repeatable decision-support workflow.

Empire's Taste addresses this gap by building a Medallion data pipeline for gastronomic trend analysis and sentiment intelligence in NYC. The project combines structured recipe information from the Spoonacular API, unstructured editorial discourse from the Eater NY RSS feed, and diner review evidence from OpenTable. OpenTable was added in Workshop 4 because the first Eater NY-only sentiment results were almost entirely positive, which made the dashboard useful for trend discovery but weak for measuring real customer satisfaction. The result is a data engineering system that transforms raw records into cleaned NLP features, governance metrics, storytelling aggregations, and dashboards for both technical and functional users.

1.2 Problem Statement

The central problem is that event menu recommendations remain largely intuition-driven. Catering managers need to know which foods are currently trending, whether those foods are discussed positively or negatively by both media and diners, and whether matching recipes satisfy event and dietary requirements. Without an automated pipeline, this information remains fragmented across APIs, RSS feeds, articles, reviews, and manual analysis.

1.3 Course Project Evolution

The project was developed incrementally through four workshops and consolidated in the final paper and poster. Workshop 1 established the data analysis foundation, candidate sources, user stories, and early validation samples. Workshop 2 defined the technical architecture and implemented the Bronze/Silver pipeline in Docker and Airflow. Workshop 3 extended the system with data profiling, quality assessment, NLP cleaning, and Silver-to-Gold preparation. Workshop 4 completed the Gold layer, added OpenTable as a diner-review source, corrected

the sentiment endpoint that had been dominated by positive Eater NY coverage, delivered the governance and storytelling dashboards, and finalized validation. The final poster and paper synthesize these increments into the complete Empire's Taste solution.

Chapter 2

Literature Review

2.1 Opinion Mining and Review Summarization

Opinion mining provides the theoretical foundation for converting large quantities of subjective text into decision-support signals. Hu and Liu (6) argue that the value of review mining is not only to classify an entire document as positive or negative, but to identify the specific features or attributes that customers discuss and then summarize the polarity attached to those features. Their feature-based review summarization process involves three steps: extracting the product features mentioned by users, identifying opinion-bearing sentences, and producing a summary of positive and negative evidence for each feature.

This idea is directly relevant to Empire's Taste because catering managers do not only need to know whether the general NYC food conversation is positive. They need to know which dishes, ingredients, cuisines, restaurants, or event-relevant attributes are being discussed positively or negatively. The project's keyword sentiment, named-entity extraction, review sentiment, and source-comparison aggregations follow the same principle of moving from raw text to feature-level summaries. Instead of summarizing digital camera attributes, the pipeline summarizes gastronomic attributes such as omakase, bakery, ramen, dietary labels, neighborhoods, and restaurants.

2.2 Sentiment Analysis Foundations

Pang and Lee (9) define opinion mining and sentiment analysis as the computational treatment of opinions, sentiments, evaluations, attitudes, and emotions in text. Their survey highlights several challenges that are important for this project: sentiment analysis differs from factual information retrieval because the same term may carry different meanings depending on context; polarity can vary by target; and subjectivity detection is necessary because not every sentence in a document expresses an opinion.

These challenges appear in culinary media and restaurant reviews. An article may mention a restaurant closure factually, praise one dish, criticize service, and describe a neighborhood without sentiment. Similarly, OpenTable review text may include food-specific praise mixed with generic comments about staff or ambience. For this reason, the pipeline does not treat all tokens as useful trend signals. It cleans text, removes domain stopwords, extracts food-oriented keywords, separates named entities from culinary terms, and computes sentiment at the cleaned text level before aggregating results for dashboards.

2.3 VADER and Short-Text Sentiment

Hutto and Gilbert (7) propose VADER as a parsimonious rule-based model for sentiment analysis in short, informal, and social-media-like text. Their work is important because it combines a validated sentiment lexicon with rules for punctuation, capitalization, degree modifiers, contrastive conjunctions, and negation. The model outputs a compound sentiment score that can be interpreted using conventional thresholds for positive, neutral, and negative polarity.

VADER is appropriate for Empire's Taste for three reasons. First, Eater NY summaries and OpenTable reviews are relatively short compared with long-form documents. Second, the project needs an interpretable model whose outputs can be explained to non-technical stakeholders through dashboards. Third, VADER is lightweight enough to run in the batch Airflow pipeline without requiring model training, GPUs, or large labeled food-domain datasets. The limitation is that VADER is lexicon-based and may miss subtle culinary language, sarcasm, or domain-specific evaluations. The report therefore recommends future experimentation with domain-adapted transformer models while keeping VADER as a practical baseline.

2.4 Context-Aware and Hybrid Recommendation

Event menu recommendation is not a generic top-N recommendation problem. Context-aware recommender systems incorporate situational variables such as time, location, event type, and user constraints into recommendation logic (1). In this project, context appears through NYC geography, daily culinary trends, event-planning needs, dietary restrictions, and source-specific evidence from media, reviews, and recipes.

Hybrid recommender systems combine multiple recommendation strategies or information sources to improve decision quality (3). Empire's Taste follows this hybrid logic by combining content-based recipe metadata, editorial trend signals, review sentiment, dietary fields, and governance quality metrics. The system does not yet implement a full collaborative filtering model because it does not maintain long-term individual user histories. However, it implements the architectural foundation for hybrid recommendation by exposing aligned recipe, article, review, keyword, and sentiment features in the Gold layer.

2.5 Heterogeneous Information Sources for Recommendation

Zhang et al. (12) argue that recommender systems benefit from information sources beyond ratings, including textual reviews, images, and other heterogeneous signals. Their joint representation learning framework shows that different information sources can capture complementary aspects of preferences and can be integrated for top-N recommendation. Although Empire's Taste does not train joint neural embeddings, it follows the same conceptual direction: no single source is sufficient for menu intelligence.

Spoonacular provides recipe supply and dietary metadata, Eater NY provides editorial demand and trend discourse, and OpenTable provides diner satisfaction signals. The Gold source-comparison aggregation exists because these sources can disagree. A dish may appear frequently in the press but have weak recipe availability, or a restaurant may be praised editorially while diner reviews reveal mixed satisfaction. The literature on heterogeneous recommendation supports this multi-source design and justifies the project's separation of media, review, and recipe evidence before merging them into dashboard-ready outputs.

2.6 Data Engineering and Medallion Architecture

Data science practice frames data work as a process that combines acquisition, cleaning, transformation, analysis, interpretation, and communication (11). This aligns with the Medallion architecture used in Empire's Taste, where raw source fidelity is preserved in Bronze, analytical usability is created in Silver, and stakeholder-facing insight is produced in Gold. The Medallion pattern separates ingestion, cleaning, and serving responsibilities, which improves traceability and allows failed Gold computations to be recomputed without downloading source data again (4).

Apache Airflow supports this architecture by scheduling tasks, expressing dependencies, retrying failures, monitoring execution, and coordinating sensors (2). Pandas is suitable for source-specific Silver cleaning because it simplifies JSON normalization, null handling, list flattening, and text preprocessing (8). PySpark is appropriate for Gold because it reads folder-level Parquet datasets, resolves schema drift, and produces repeatable aggregations for dashboards and APIs.

2.7 Research Gap and Project Positioning

The reviewed literature supports each component of the project but also reveals a practical integration gap. Opinion mining research explains how to summarize feature-level sentiment, VADER provides an interpretable short-text sentiment method, recommender-system research supports context-aware and heterogeneous evidence, and data engineering literature supports layered pipelines. However, these areas are often treated separately.

Empire's Taste addresses this gap by integrating them into one reproducible pipeline for NYC gastronomic intelligence. Its contribution is not a new sentiment algorithm or recommender model; rather, it is an end-to-end architecture that operationalizes existing methods for a specific decision context: event menu planning. The system converts current culinary media, diner reviews, and recipe metadata into governed, auditable, and user-facing insights for catering managers, event planners, restaurant managers, and data analysts.

Chapter 3

Objectives, Scope, Assumptions, and Limitations

3.1 Objectives

The main objective of Empire's Taste is to demonstrate how a reproducible data engineering pipeline can support evidence-based event menu planning in New York City. The report aims to show not only that food-related data can be collected, but also that heterogeneous sources can be governed, cleaned, analyzed, and translated into practical recommendations for catering and event stakeholders.

The specific project objectives are:

1. Design and implement a Medallion data pipeline that ingests structured recipe data from Spoonacular, editorial food discourse from Eater NY, and diner review signals from OpenTable.
2. Reduce the bias of an Eater NY-only sentiment view by adding OpenTable reviews as a direct consumer-satisfaction source.
3. Apply NLP techniques, including TF-IDF, spaCy entity extraction, and VADER sentiment scoring, to identify gastronomic keywords, named entities, and positive, neutral, or negative opinion patterns.
4. Produce Gold-layer data products for recipe recommendation, sentiment storytelling, source comparison, review analysis, and data governance.
5. Expose the curated outputs through dashboards, FastAPI endpoints, and a user-facing web layer so that catering managers, event planners, restaurant managers, and data analysts can consume insights without inspecting raw code or files.
6. Validate the system through workshop-based evidence, including ingestion samples, data-quality metrics, dashboard outputs, API responses, and final paper/poster results.

These objectives are guided by the following research and engineering questions:

Q1. Which dishes, ingredients, cuisines, restaurants, or food concepts are currently visible in NYC culinary discourse?

Q2. Does consumer review sentiment confirm or challenge the positive tone found in editorial food coverage?

Q3. How can article summaries and diner reviews be transformed into reliable trend and sentiment indicators?

Q4. Which Spoonacular recipes best match the detected trends while preserving event-relevant dietary attributes?

Q5. What data-quality evidence is needed before stakeholders can trust dashboard recommendations?

3.2 Scope

The scope of the project is a course-level, end-to-end prototype for batch gastronomic intelligence and event-oriented menu recommendation. It includes the data engineering path from ingestion to stakeholder-facing presentation, with emphasis on traceability, reproducibility, and practical decision support.

The project includes:

- Source ingestion from Eater NY RSS articles, OpenTable diner reviews, and Spoonacular recipe metadata.
- Bronze JSON storage, Silver Parquet cleaning, and Gold PySpark aggregations following the Medallion architecture.
- NLP processing for text cleaning, TF-IDF keyword extraction, spaCy named entity extraction, and VADER sentiment scoring over both article summaries and review text.
- Gold outputs for recipes, articles, reviews, governance KPIs, sentiment distribution, keyword sentiment, reviews by restaurant, source comparison, and recipe-trend alignment.
- Local Docker Compose execution with Apache Airflow orchestration, FastAPI serving, Plotly Dash dashboards, and a Next.js user interface.
- A proposed cloud implementation as a future scaling direction, supported by the cloud architecture figure and discussion.

The project excludes:

- Real-time streaming ingestion, because the implemented pipeline is batch-oriented and manually triggered for workshop validation.
- A deployed production cloud environment; the cloud architecture is proposed but not implemented as a live service.
- Direct restaurant booking, payment, inventory management, or transactional catering workflows.
- Full personalization based on long-term individual diner histories or collaborative filtering profiles.
- A production-scale commercial OpenTable integration; OpenTable is used as a prototype review signal collected through the implemented workshop pipeline.
- Formal user testing with catering managers or event planners beyond the functional user stories and dashboard validation evidence.

3.3 Assumptions

The project is based on the following assumptions:

- Spoonacular provides sufficiently structured recipe metadata, dietary tags, scores, and preparation attributes to represent recipe supply for menu-planning analysis.
- Eater NY is a useful proxy for NYC editorial culinary trends, even though its professional coverage can be more promotional and positive than consumer feedback.
- OpenTable reviews provide a useful counterweight to editorial sentiment because they contain direct diner ratings and free-text opinions about restaurants, food, service, and ambience.
- Article summaries and review texts are long enough and informative enough for TF-IDF, named entity extraction, and VADER sentiment scoring to produce usable analytical signals.
- VADER is an acceptable interpretable baseline for short culinary texts, and its compound-score thresholds are sufficient for the positive, neutral, and negative dashboard categories used in the prototype.
- Batch processing is adequate for event menu planning because catering decisions usually do not require second-by-second streaming updates.
- Local Docker execution is sufficient to validate the course project, while the proposed cloud architecture represents the expected path for production hardening.
- Stakeholders can use aggregated dashboard insights and API outputs as decision-support evidence, but final menu decisions still require human judgement, operational constraints, and domain expertise.

3.4 Limitations

Several limitations affect how the results should be interpreted. First, the data sources are not exhaustive representations of the NYC food market. Eater NY reflects editorial selection and initially produced an overly positive sentiment distribution, while OpenTable reduces that bias but still depends on restaurant availability, review volume, rating skew, and the stability of browser-session extraction. External website or API changes could break ingestion logic or modify available fields.

Second, the analytical methods are practical baselines rather than domain-specialized models. TF-IDF can identify distinctive terms but does not fully understand culinary context, popularity causality, or emerging slang. VADER is interpretable and lightweight, but it may miss sarcasm, subtle food criticism, mixed opinions within one review, and culinary expressions that differ from general sentiment language. The sentiment percentages should therefore be read as decision-support indicators rather than definitive measures of public opinion.

Third, the implementation is constrained by course time, local infrastructure, and prototype validation. The system runs in Docker Compose rather than a managed cloud platform, the DAG is manually triggered for controlled validation, and scalability results are not equivalent to production benchmarks. Some recipe fields from Spoonacular are incomplete or inconsistent, and schema drift may require defensive casting or future maintenance. Finally, the dashboards validate functional requirements through implemented user stories, but they have not been evaluated through a formal stakeholder study.

Chapter 4

Methodology

4.1 Development Methodology

The project followed an iterative workshop methodology. Each workshop added a measurable layer of functionality and validation. Workshop 1 focused on data analysis and feasibility. Workshop 2 formalized the technical architecture and Bronze/Silver implementation. Workshop 3 strengthened data quality, NLP cleaning, and profiling. Workshop 4 added OpenTable reviews, finalized Gold aggregations, corrected the sentiment view, and completed dashboard/API integration testing.

4.2 System Architecture

The architecture follows the Medallion pattern. The Bronze layer stores immutable raw JSON from Spoonacular, Eater NY, and OpenTable. The Silver layer applies cleaning, normalization, deduplication, null handling, HTML stripping, text preprocessing, and Parquet persistence. The Gold layer uses PySpark to produce curated outputs for recipes, articles, reviews, governance KPIs, and storytelling metrics.

Layer	Main Tools	Purpose
Bronze	Airflow, requests, RSS parsing, Playwright, JSON	Preserve raw Spoonacular, Eater NY, and OpenTable records with timestamps for lineage.
Silver	Pandas, text cleaning, Parquet	Clean, normalize, deduplicate, and enforce reusable analytical schemas.
Gold	PySpark, Parquet, Plotly Dash	Aggregate governance KPIs, sentiment summaries, keywords, and recipe recommendations.

Table 4.1: Medallion architecture used by Empire's Taste.

4.3 Data Sources

Spoonacular provides recipe metadata, ingredients, diets, health scores, costs, and preparation attributes. Eater NY RSS provides editorial titles, summaries, categories, publication dates, authors, URLs, and article identifiers. OpenTable review data adds a public-satisfaction signal through diner ratings for food, service, ambience, and free-text review content. Together,

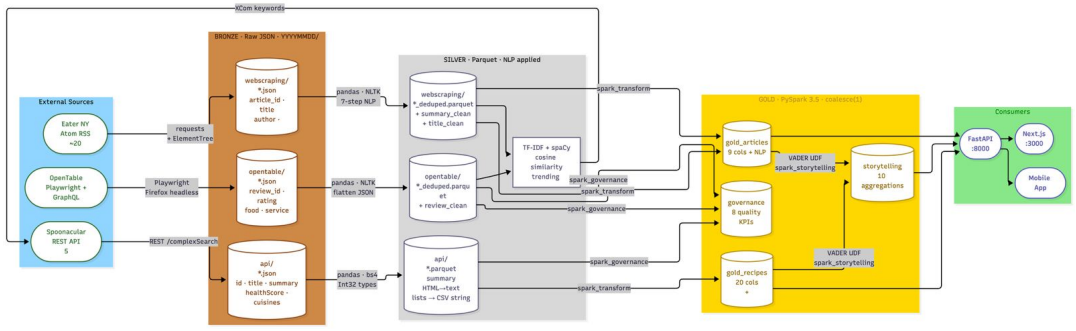

Figure 4.1: Detailed Medallion structure for the Empire's Taste pipeline.

these sources support the distinction between supply-side recipe availability, media demand, and diner satisfaction.

4.4 Pipeline Workflow

Apache Airflow orchestrates the production workflow through the unified gastronomy DAG. The DAG uses a sequential dependency chain and limits active executions so that a new run does not overwrite or conflict with the previous run. The implemented workflow covers Eater NY extraction, Eater NY cleaning, OpenTable extraction, OpenTable cleaning, trend analysis, Spoonacular search, recipe cleaning, Gold preparation, optional S3 upload, and retention cleanup.

The workflow begins with Eater NY RSS extraction because the system is designed to be trend-driven rather than keyword-static. Raw article records are written to the Bronze web-scraping folder for the current date. The Silver task cleans those records, writes deduplicated Parquet files to the matching Silver web-scraping folder, and exposes cleaned summaries for trend analysis. OpenTable reviews are then extracted and cleaned as an additional public-satisfaction signal. The trend analysis task combines cleaned article text and review text to extract food-related keywords, then passes those keywords through Airflow XCom. Those keywords are used by the Spoonacular ingestion task to query recipes that match current culinary discussion. Recipe records are written to the Bronze API folder, cleaned into the Silver API folder, and finally consolidated by PySpark into Gold outputs.

Each layer writes to a date-partitioned directory using the `YYYY-MM-DD` convention. This design preserves history because new runs accumulate rather than overwriting previous outputs. Downstream consumers read the latest available date directory, which allows dashboards and API endpoints to remain stable even when earlier runs are retained for auditability.

4.5 Airflow, API, and Retention Design

The Airflow/API architecture separates orchestration into standalone and unified execution modes. The standalone Bronze DAG is used for ad-hoc ingestion of Eater NY and OpenTable content. The Silver processing DAG waits for recent Bronze partitions, cleans article and review data, analyzes trends, queries Spoonacular, and cleans recipe data. The Gold processing DAG waits for the Silver branches and then executes Spark transformations, governance computation, and storytelling aggregation. The unified DAG chains the complete production path and adds two operational tasks: uploading the latest Gold partition to S3 when a bucket is configured and deleting old partitions according to a retention window. In Workshop 4 the

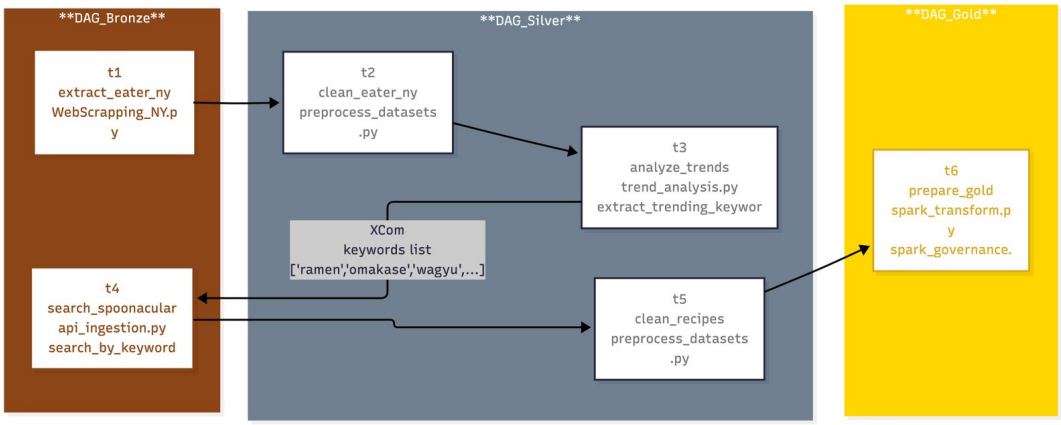

Figure 4.2: DAG sequence used to orchestrate the trend-driven ingestion, cleaning, API search, and Gold preparation workflow.

DAGs were configured for manual triggering to avoid accidental reruns while validating the browser-based OpenTable extraction.

Sensors were implemented as Python-based checks over date-partitioned folders rather than brittle environment-specific filesystem connections. Silver sensors use short polling to detect new Bronze JSON records, while Gold sensors use longer timeouts because Spark outputs depend on API, web, and review branches. Gold and unified pipelines use single active runs to avoid concurrent Spark JVM conflicts. Each Spark task opens and closes its own SparkSession, reducing the risk of memory contention and stale JVM state.

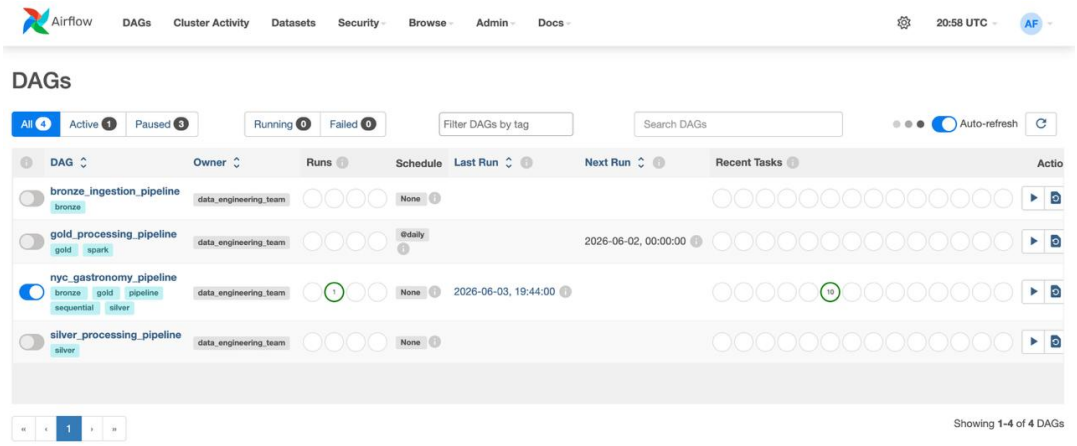

Figure 4.3: Airflow interface showing the executed pipeline and task monitoring evidence.

The API methodology is based on a loader abstraction. In local Docker execution, the loader reads the latest Gold partition from the filesystem. In production-oriented scenarios, the same pattern can read from S3 using the exported Gold partition. The loader is responsible for locating the latest date directory, handling Spark coalesced Parquet directories, and exposing reusable loading functions for storytelling, governance, recommendations, and partition metadata. This design keeps the FastAPI routers independent from the physical storage backend.

4.6 Web and Mobile Client Methodology

The web architecture documentation adds a user-facing layer above the data pipeline. The Next.js 15 application uses the App Router, React components, TailwindCSS, NextAuth with Google OAuth, Prisma, and SQLite. The frontend does not execute ETL; it consumes FastAPI endpoints and focuses on authenticated navigation, dashboard rendering, favorites management, and recipe exploration. The dashboard routes are protected by middleware and verified again on the server through NextAuth and Prisma-backed sessions.

The web data-access layer centralizes HTTP calls in a typed API client. Server-side calls use an internal API URL when the web app and FastAPI service run in Docker, while browser-side calls use the public API URL. Requests include timeout handling and normalized error behavior so the UI can show fallback states when a dashboard module lacks data. The mobile architecture follows the same principle: the Expo client consumes FastAPI Gold-layer endpoints, presents trends, recipes, favorites, and data-quality views, and requires the backend to be reachable from the same local network during development.

4.7 NLP and Analytics Methods

The NLP workflow removes photo credits, HTML entities, URLs, boilerplate, punctuation, numeric noise, short tokens, and domain-specific stopwords. TF-IDF identifies distinctive culinary keywords for the current batch, while spaCy semantic vectors compare candidate terms against a gastronomic anchor vocabulary so that food concepts are separated from generic words such as service, staff, or experience. A curated cuisine vocabulary preserves terms that vectors may miss, such as omakase, birria, and vegan. VADER computes sentiment polarity over cleaned article summaries and OpenTable review text using the common compound-score thresholds: positive at ≥ 0.05, neutral between -0.05 and 0.05, and negative at ≤ -0.05. PySpark then aggregates these outputs into sentiment distributions, keyword rankings, trend windows, governance KPIs, and recipe-storytelling datasets.

4.8 Course Workshop Traceability

Stage	Focus	Evidence Integrated in Report
Workshop 1	Data analysis, source selection, user stories, and early samples.	Spoonacular and Eater NY feasibility, functional users, recommendation questions, and later evidence motivating OpenTable.
Workshop 2	Technical architecture and Bronze/Silver pipeline.	Docker Compose, Airflow DAGs, JSON Bronze storage, Parquet Silver outputs.
Workshop 3	Data profiling, quality findings, and NLP cleaning.	Missingness, schema drift, duplicate handling, text preprocessing, quality rules.
Workshop 4	Gold layer, OpenTable review integration, governance dashboard, storytelling dashboard, and integration fixes.	PySpark outputs, combined article/review sentiment, KPI mapping, dashboard design, final validation.
Final paper/poster	Consolidated presentation and academic framing.	Abstract, literature framing, final architecture, conclusions, and future work.

Table 4.2: Traceability between course workshops and the final report.

Chapter 5

Implementation and Results

5.1 Implemented Solution

The implemented solution is a local end-to-end data pipeline named Empire's Taste. It runs in a Docker Compose environment, uses Apache Airflow for orchestration, stores raw files in a Bronze data lake, writes cleaned Parquet files to Silver, and produces Gold Parquet outputs consumed by two Plotly Dash dashboards, a FastAPI serving layer, and a Next.js 15 web application. The implementation reflects the repository organization described in the README: Airflow DAGs and processing scripts are placed under `airflow/dags/`, dashboard applications under `dashboard/`, API endpoints under `api/`, and data products under `datalake_bronze/`, `datalake_silver/`, and `datalake_gold/`.

5.1.1 Repository-Level Implementation

The Airflow implementation is centered on `nyc_gastronomy_pipeline.py`, the main unified DAG. Standalone DAGs for Bronze, Silver, and Gold processing remain available for modular execution, but the final workflow uses the unified DAG to demonstrate the complete path from ingestion to dashboards. The Bronze scripts include `api_ingestion.py`, which acts as the Spoonacular REST client; `WebScrapping_NY.py`, which extracts Eater NY RSS content; and `opentable_ingestion.py`, which uses Playwright and OpenTable's review endpoint to collect diner feedback. The Silver scripts include `preprocess_datasets.py`, responsible for source-specific cleaning and Parquet writing, and `trend_analysis.py`, responsible for keyword extraction and trend detection across articles and reviews. The Gold scripts include `spark_transform.py`, `spark_governance.py`, and `spark_storytelling.py`, which consolidate recipes, articles, and reviews, compute quality KPIs, and prepare storytelling aggregations.

The dashboard folder implements two separate Plotly Dash applications. The governance application presents data quality and pipeline-health indicators for technical users. The storytelling application presents sentiment, trend, keyword, and source-comparison insights for chefs and event planners. The Next.js user interface provides routes for trends, recipes, favorites, and data quality. The FastAPI layer exposes Gold outputs through governance, storytelling, recommendations, sentiment, keyword, trend, entity, source, and summary routes, allowing the same curated data products to support dashboards, the web app, and programmatic clients.

5.1.2 Execution Flow

The execution flow is intentionally sequential. First, the pipeline extracts Eater NY records. Second, it cleans and deduplicates the article data. Third, it extracts OpenTable reviews. Fourth, it flattens, cleans, and deduplicates review records by review identifier. Fifth, it analyzes cleaned article and review text to identify the food terms currently driving public discourse. Sixth, those terms are passed through Airflow XCom into the Spoonacular API search task. Seventh, Spoonacular responses are cleaned and normalized. Eighth, PySpark builds Gold outputs for recipes, articles, reviews, governance, and storytelling. This sequence is important because the recipe search depends on trends extracted from both editorial coverage and diner feedback; therefore, recipes are not selected from a fixed list but from the current culinary context.

5.1.3 Data Products

The final Gold layer writes artifact families for recipe details, cleaned articles with sentiment, review sentiment, governance KPIs, and storytelling aggregations. The recipe artifact supports recommendation and filtering. The article and review artifacts preserve the cleaned textual evidence and sentiment scores. The governance artifact supports operational trust through metrics such as null rate, duplicate rate, record count, outlier rate, schema compliance, NLP compression, non-ASCII character ratio, and timeliness. The storytelling artifact supports non-technical interpretation through ten aggregation families: sentiment distribution, sentiment trend, top keywords, keyword sentiment, recipe-trend alignment, review sentiment, reviews by restaurant, named entities, dietary breakdown, and source comparison.

The dashboard storytelling layer defines human-readable trend labels. Keywords with high TF-IDF scores and positive sentiment are labeled as highly trending, trending, or growing, while high-scoring negative keywords can be labeled as heavily criticized. This converts raw numerical outputs into language that chefs and event planners can interpret quickly.

5.1.4 Operational Services and Configuration

The implementation is deployed through Docker Compose. The core services are the Airflow scheduler, Airflow webserver, FastAPI service, web frontend, governance dashboard, storytelling dashboard, and PostgreSQL metadata database. The Airflow services use the project environment file for variables such as the Spoonacular API key, optional API key rotation values, retention settings, and optional S3 export configuration. The web service uses variables such as `NEXT_PUBLIC_API_URL`, `API_INTERNAL_URL`, Google OAuth credentials, NextAuth secrets, and `DATABASE_URL` for Prisma/SQLite persistence.

The data lake folders are mounted as shared volumes so that Airflow can write Bronze, Silver, and Gold outputs while FastAPI and dashboards read curated Gold data. The DAG folder is also mounted for hot reload, allowing DAG and script updates without rebuilding the entire Airflow image. In contrast, Dockerfile or dependency changes require rebuilding the relevant image. This distinction was important during implementation because missing PySpark or NLP dependencies were resolved by rebuilding images rather than only restarting containers.

5.1.5 FastAPI Serving Implementation

FastAPI turns Gold data into a service layer. Governance routes expose KPI summaries, null rates, and outlier information. Storytelling routes expose sentiment distribution, top keywords,

temporal trends, named entities, source summaries, and narrative summaries. Recommendation routes expose trending recipes and recipe detail retrieval. The loader always attempts to serve the most recent Gold partition, which protects the UI from failing when the latest manual run has not yet produced a current partition.

The API is intentionally downstream-only: it does not call Spoonacular, scrape Eater NY, or recompute Spark aggregations at request time. This preserves the Medallion separation of responsibilities. Heavy ingestion, NLP, and aggregation occur in Airflow and Spark; FastAPI performs lightweight loading, filtering, serialization, and response shaping.

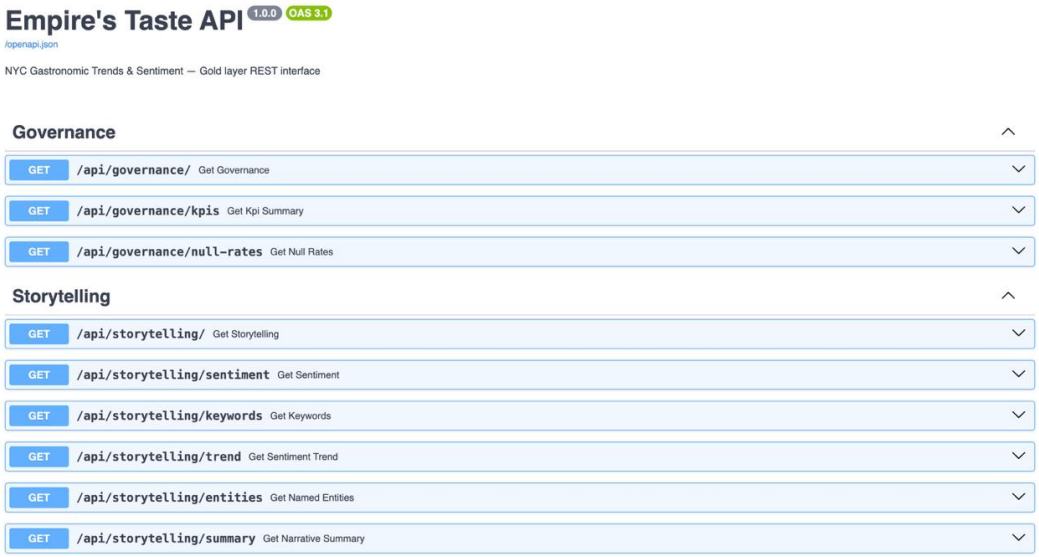

Figure 5.1: FastAPI endpoint documentation exposing Gold-layer governance, storytelling, recommendation, and analytics routes.

5.1.6 Web and Mobile Implementation

The web implementation contains a public landing page, Google login page, authenticated dashboard shell, and four primary dashboard modules: trends, recipes, favorites, and data. The trends module renders sentiment, keywords, trend lines, and entities, often in parallel server-side calls. The recipes module supports reactive filters by cuisine, diet, preparation time, and search text. The favorites module persists saved recipes per user and prevents duplicates through a unique user-recipe key. The data module renders governance KPIs, null rates, layer status, and user-readable quality flags.

The mobile client mirrors the same product logic for Expo. It consumes the FastAPI backend over the local network, shows a sentiment index, temporal sentiment chart, top keywords, named entities, recipe filters, saved plates, and data-quality indicators. This mobile layer was not the central course deliverable, but it demonstrates that the Gold/API contract can support multiple client experiences without changing the pipeline.

5.2 Workshop 1 Results: Data Analysis Foundation

Workshop 1 validated the feasibility of combining recipe, editorial, and review-oriented sources. Spoonacular supplied structured recipe fields, including dietary attributes such as vegetarian

and gluten-free flags, diets, dish types, preparation metadata, servings, price, health score, and Spoonacular score. Eater NY RSS supplied article titles, summaries, publication dates, categories, authors, URLs, and identifiers. This stage confirmed that the project could connect structured recipe supply with unstructured market discourse, while later validation showed that a direct diner-review source was necessary to avoid an overly promotional sentiment view. It also defined the functional users and user stories that later guided the dashboards: catering managers needed competitive dish shortlists, event planners needed dietary filtering, restaurant managers needed sentiment monitoring, and data analysts needed quality metrics.

5.3 Workshop 2 Results: Bronze and Silver Pipeline

Workshop 2 produced the first functional architecture and ingestion workflow. The Bronze layer saved datetime-stamped JSON files for traceability under source-specific folders. The Silver layer transformed raw records into cleaned Parquet outputs using Pandas and schema-oriented processing. For API records, list fields such as ingredients, cuisines, and diets were flattened into reusable strings, HTML was stripped from recipe summaries, time fields were coerced into numeric types, and recipes were deduplicated by identifier. For web records, article titles and summaries were normalized, punctuation and stopwords were removed, and article records were deduplicated by URL or article identifier. Airflow provided task scheduling, monitoring, retries, and dependency management inside Docker.

5.4 Workshop 3 Results: Data Quality and NLP Processing

Workshop 3 identified and addressed quality issues. Missing preparation and cooking time fields were preserved as nullable values instead of artificial sentinel values. Duplicate article batches were handled through ingestion checks and Silver deduplication by article identifier. Regex-based cleaning removed HTML entities, attribution prefixes, URLs, and non-informative tokens from article summaries. The notebook evidence also explored null-rate analysis, duplicate detection, text length statistics, outlier rates using the IQR method, and preliminary Gold governance and storytelling views. These analyses directly informed the Gold KPI schema and dashboard design.

5.5 Workshop 4 Results: Gold Layer, API, and Web Experience

Workshop 4 completed the Gold layer, OpenTable review integration, dashboard interfaces, API serving layer, and user-facing web experience. PySpark generated recipe artifacts, article artifacts, review artifacts, governance KPIs, and storytelling metrics. The OpenTable addition addressed a concrete analytical problem: Eater NY article summaries alone produced a sentiment distribution that was effectively 100% positive, while combining article sentiment with review sentiment produced a more realistic distribution of 89% positive, 7.65% negative, and 3.28% neutral records in the validated run. The Governance Dashboard at `localhost:8050` surfaces pipeline health using total records, null rates, duplicate rates, schema compliance, outlier charts, volume charts, and audit tables. The Storytelling Dashboard at `localhost:8051` translates NLP results into plain-language insights for chefs and event planners through a narrative card, sentiment donut, sentiment trend, top keyword chart, media-versus-kitchen comparison, activity chart, and contextual mentions. The final implementation also adds FastAPI access to the same Gold data, making the architecture usable by dashboards and programmatic clients. The API exposes storytelling endpoints for sentiment, keywords, trends,

entities, sources, and summary data; recommendation endpoints for filtered recipe retrieval; and governance endpoints for KPIs, null rates, and outliers. Its loader reads the most recent Gold partition so that the application can still serve the latest available data if the current manual run has not completed.

5.5.1 Deployment and User Interface Evidence

The Docker deployment runs as a seven-service environment: Airflow scheduler, Airflow webserver, FastAPI service, Next.js web application, governance dashboard, storytelling dashboard, and PostgreSQL metadata database. Shared volumes mount the Bronze, Silver, and Gold data lake folders so that Airflow writes data products and the API/dashboard services read the same outputs. The deployment supports hot-reload behavior for Airflow DAGs, while dependency or Dockerfile changes require container rebuilds.

The Next.js web application extends the analytical outputs into user-facing pages. The trends page presents the keyword of the day, combined positive sentiment percentage, mood label, trend board, sentiment donut, weekly trend line, and named entities. The recipes page presents Spoonacular recipes selected from the current article-and-review keywords and supports filters for cuisine, diet, and preparation time. The favorites page stores a personal recipe collection for authenticated users using Google OAuth, Prisma, and SQLite. The data page translates technical KPIs into user-readable labels such as records collected, missing data, and data quality, while also linking article titles back to Eater NY and exposing OpenTable review context.

5.5.2 Validation Evidence

The project and architecture documents define a reproducible operational workflow. A developer starts Docker Desktop, configures the root environment file, builds the stack, opens Airflow, manually triggers the unified gastronomy DAG, and then validates the generated services through the Airflow UI, FastAPI Swagger documentation, dashboards, and web/mobile clients. This evidence matters because the implementation is not only a set of scripts; it is a runnable multi-service system with documented ports, credentials, environment variables, and troubleshooting steps.

The documentation also records important operational safeguards. The pipeline keeps date-partitioned history instead of overwriting data. The retention task removes partitions older than the configured window, reducing local storage growth. Optional S3 upload prepares the latest Gold output for production-style API consumption. The API loader can serve the latest available partition even if today's DAG has not completed. These behaviors make the implementation more robust than a one-time notebook workflow.

5.6 Data Quality Results

Silver profiling found that `preparationMinutes` and `cookingMinutes` had a 91.3% null rate, while `description` had a 95.7% null rate. A schema drift issue was detected in the Spoonacular `license` field, which appeared with inconsistent types and was safely cast to string in Gold processing. Numeric outliers in likes, price, and scores were retained because they can represent valid business cases.

The Gold governance dataset stores KPI rows in long format for completeness, volume, uniqueness, validity, accuracy, timeliness, schema compliance, outlier detection, NLP compression, and non-ASCII character noise. This long-format design uses fields such as KPI name,

category, source, field, value, unit, finding, and computation timestamp so that any analytical tool can consume the metrics without reshaping. The NLP compression ratio averaged 0.677, indicating that cleaning reduced noisy text while preserving meaningful content.

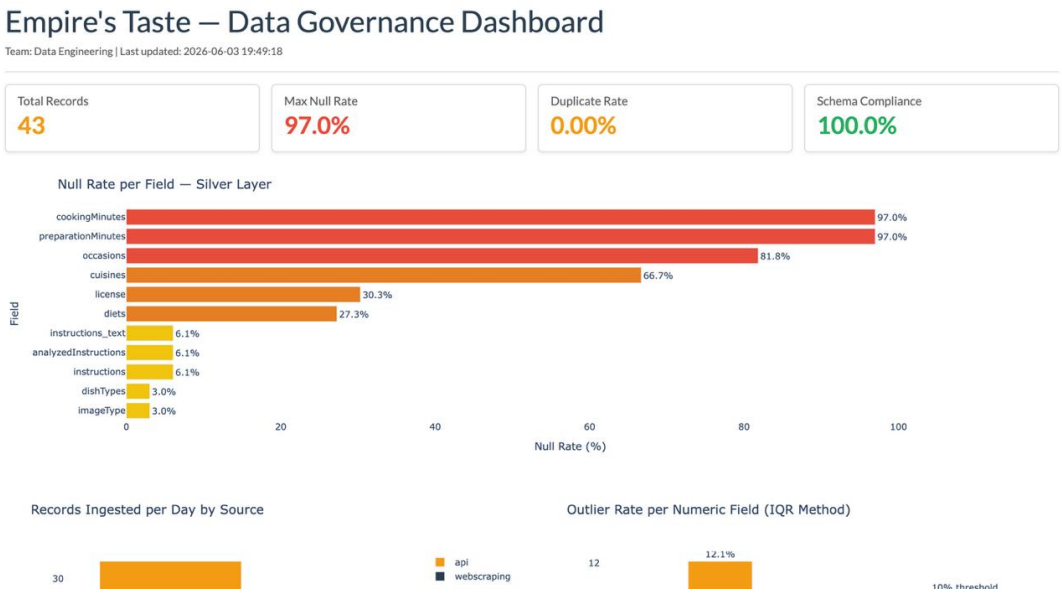

Figure 5.2: Null analysis evidence used to validate missingness patterns and inform governance KPIs.

5.7 Functional Results

The final dashboards, Next.js web pages, and API endpoints answer the core user stories defined during project development. Catering managers can inspect positive trend topics and recipe candidates. Event planners can filter dish ideas by dietary tags. Restaurant managers can monitor sentiment changes over time. Data analysts can inspect record counts, duplicate rates, null rates, and schema compliance before trusting downstream recommendations.

In practical terms, the user histories were successfully implemented because each one was translated into a concrete data product or interface component. The catering manager story is implemented by the trend-driven recipe flow: article and review keywords are extracted first and then passed to Spoonacular, so recipe recommendations follow current NYC discussion rather than a static keyword list. The event planner story is implemented by retaining dietary attributes through Silver and Gold rather than dropping them during cleaning. The restaurant manager story is implemented by VADER sentiment scores and temporal Gold aggregations that combine editorial sentiment with OpenTable review sentiment for a less promotional view of public mood. The data analyst story is implemented by the separate governance dashboard and FastAPI route, which expose data quality checks independently from business storytelling. The lower-priority alert and source-comparison stories were partially implemented as analytical foundations: the Gold layer computes the sentiment trends and source comparisons required for alerts, while real-time notifications remain future work.

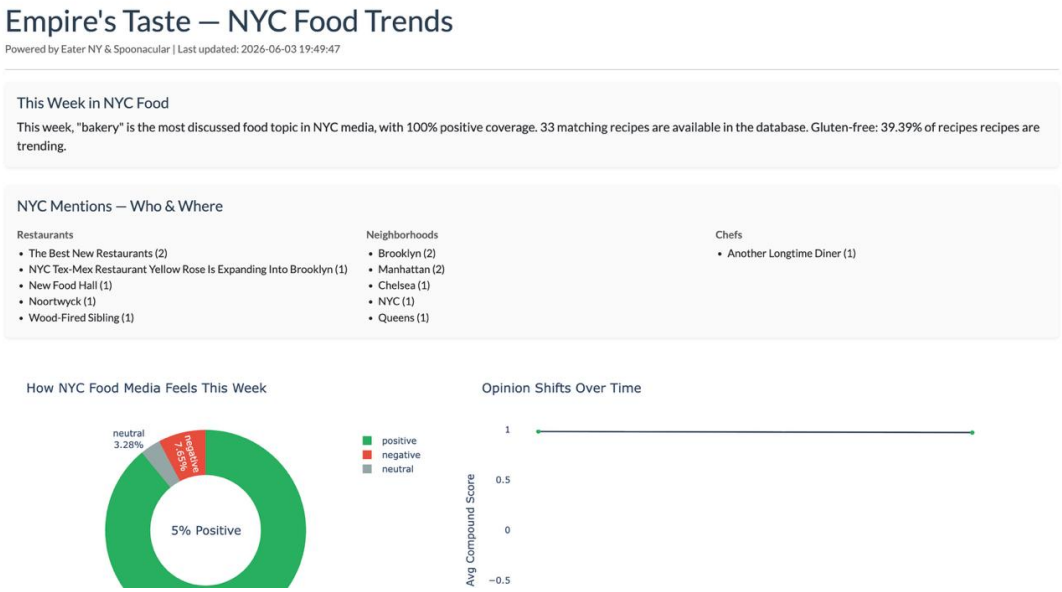

Figure 5.3: Sentiment analysis dashboard evidence showing how cleaned text is translated into functional insight.

Workshop 1 User Story	Implementation Evidence	Delivered Value
Catering manager sees highest-rated dishes for an event.	Trend keywords from Eater NY and OpenTable reviews drive Spoonacular searches; Gold recipe outputs rank candidate recipes using recipe scores and trend alignment.	Replaces intuition with a current, evidence-based dish shortlist.
Event planner filters dishes by dietary restriction.	Silver recipe cleaning preserves diet fields such as vegan, vegetarian, gluten-free, dairy-free, and related Spoonacular attributes.	Supports inclusive menu proposals for mixed guest profiles.
Restaurant manager monitors sentiment trends over time.	VADER scores cleaned article summaries and OpenTable reviews; Gold storytelling outputs sentiment distribution and temporal sentiment trend aggregations.	Shows whether cuisines, dishes, or formats are gaining positive or negative attention.
Data analyst views data quality metrics.	Gold governance outputs null rate, duplicate rate, record count, schema compliance, outlier rate, and NLP compression KPIs.	Provides trust and auditability before recommendations are used.
Event planner identifies neighborhood or context mentions.	spaCy entity extraction and keyword aggregation surface restaurants, neighborhoods, people, and food terms from NYC articles.	Connects menu ideas to local context and event themes.
Manager receives future alert candidates for sentiment drops.	Keyword-sentiment association and sentiment trend outputs expose falling compound scores, even though automated alerting remains future work.	Establishes the analytical base for proactive menu risk management.
Analyst compares media, recipe, and review signals.	Source-comparison storytelling outputs contrast article activity with recipe availability, recipe quality indicators, and OpenTable review sentiment.	Helps validate whether public buzz, recipe supply, and diner satisfaction tell a consistent story.

Table 5.1: Implementation of Workshop 1 user stories in the final system.

Chapter 6

Discussion

6.1 Interpretation of Results

The results show that a Medallion pipeline is appropriate for transforming heterogeneous culinary data, including press articles, diner reviews, and recipe metadata, into usable recommendation intelligence. Bronze storage preserves raw lineage, Silver processing makes the data reliable, and Gold aggregations make insights accessible. The separation between governance and storytelling dashboards is important because it serves both engineering trust and business usability.

6.2 Alignment with Objectives

The project meets its main objectives by implementing ingestion, NLP processing, sentiment scoring, governance metrics, dashboards, API serving, and a user-facing web layer. The workshop sequence demonstrates progressive maturity: data feasibility and user histories in Workshop 1, architecture and ingestion in Workshop 2, quality/NLP hardening in Workshop 3, and final dashboard delivery in Workshop 4. The implementation is therefore not only technically complete but also traceable to the original functional requirements.

6.3 Technical Challenges

Major challenges included empty API responses, duplicate RSS batches, missing recipe fields, noisy article and review text, OpenTable session extraction, filesystem sensing inside containers, Spark Row access errors, dependency synchronization, and schema drift. These were mitigated through defensive checks, nullable typing, deduplication rules, regex cleaning, Python-based file sensing, attribute-based Spark Row access, and Docker image rebuilds.

6.4 Practical Implications

For catering managers, the solution reduces reliance on intuition by connecting menu proposals to current culinary discourse. For data analysts, it provides auditable quality signals before insights are presented. For event planners, it supports more inclusive recommendations by linking trend-aware recipes to dietary constraints.

6.5 Scalability and Future Improvements

The current implementation is suitable for course validation and local execution, but the next engineering step is to move the architecture from a local Docker Compose prototype to a managed cloud implementation. In that version, ingestion services would run as scheduled cloud jobs or managed workflow tasks, raw Eater NY, OpenTable, and Spoonacular data would be stored in cloud object storage as the Bronze layer, and Silver and Gold transformations would run on managed Spark or serverless data-processing services. The curated Gold outputs would then be exposed through a deployed FastAPI service and consumed by the web dashboard, mobile client, and analytics dashboards without depending on a developer laptop.

A cloud deployment would also improve reliability and governance. Object storage would preserve partitioned history, managed orchestration would provide retries and monitoring, secrets would be stored in a cloud secret manager rather than local environment files, and dashboard/API services could scale independently from batch processing. This design would make it easier to add automated alerting for sentiment drops, scheduled data-quality checks, and controlled access for catering managers, event planners, and analysts.

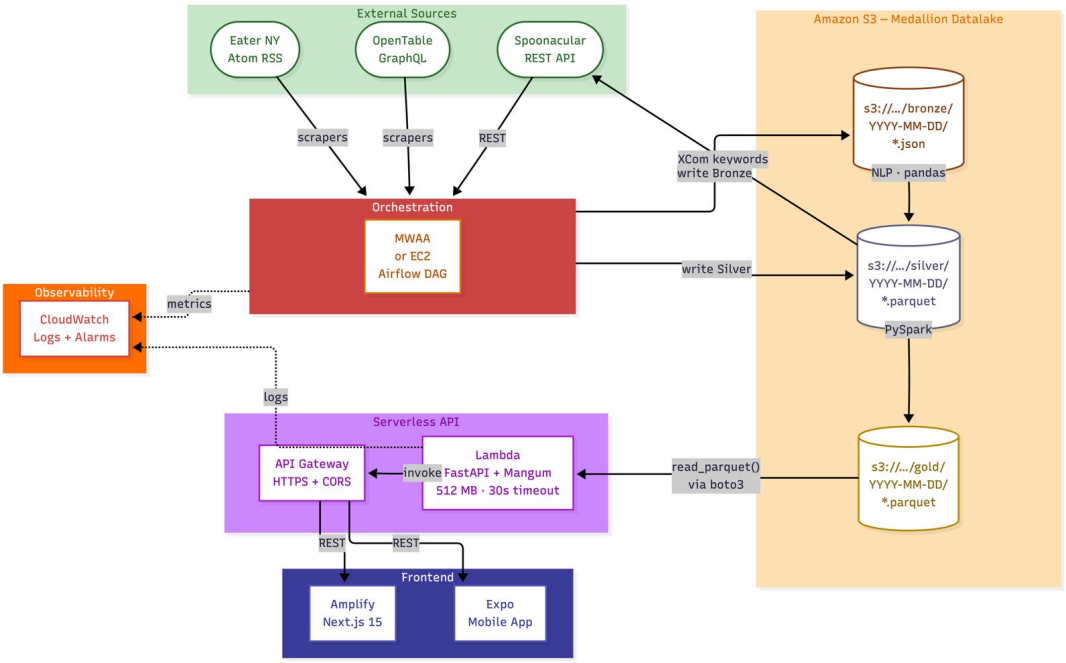

Figure 6.1: Proposed cloud implementation structure for scaling the Empire's Taste pipeline beyond local Docker execution.

Future work should therefore harden and expand the OpenTable-style review ingestion, add more direct consumer review sources, expand Gold aggregations to rolling multi-week windows, implement the proposed cloud architecture, and evaluate transformer-based sentiment models for more nuanced food journalism and diner feedback.

Chapter 7

Conclusions

7.1 Summary of Findings

Empire's Taste demonstrates that structured recipe data, unstructured culinary media, and diner reviews can be integrated into a single event-oriented recommendation pipeline. The system successfully applies a Medallion architecture, Airflow orchestration, NLP processing, PySpark aggregation, and Plotly Dash visualization to convert raw food discourse and customer feedback into actionable menu insights.

7.2 Conclusions

The project confirms that data engineering can support more systematic menu planning in competitive gastronomic environments. The final system provides technical governance for reliability and functional storytelling for decision-making. By incorporating all course workshops, the report shows a clear progression from data exploration to final dashboards and academic presentation.

7.3 Recommendations

Future versions should harden the review ingestion process for production use, implement the proposed cloud deployment, strengthen sentiment analysis with food-domain transformer models, add alerting for sudden sentiment drops, deploy the API loader against cloud object storage, and extend the dashboard with neighborhood-specific and event-specific recommendation views.

References

[1] Adomavicius, G. and Tuzhilin, A. (2011) 'Context-aware recommender systems', in *Recommender Systems Handbook*. Springer, pp. 217–253.

[2] Apache Software Foundation (2026) *Apache Airflow Documentation*. Available at: https://airflow.apache.org/docs/ (Accessed: 27 March 2026).

[3] Burke, R. (2002) 'Hybrid recommender systems: Survey and experiments', *User Modeling and User-Adapted Interaction*, 12(4), pp. 331–370.

[4] Databricks (2026) *Medallion Architecture*. Available at: https://www.databricks.com/glossary/medallion-architecture (Accessed: 27 March 2026).

[5] Explosion AI (2026) *spaCy — Industrial-strength NLP*. Available at: https://spacy.io (Accessed: 27 March 2026).

[6] Hu, M. and Liu, B. (2004) 'Mining and summarizing customer reviews', in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Seattle, WA, pp. 168–177.

[7] Hutto, C. J. and Gilbert, E. (2014) 'VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text', in *Proceedings of the 8th International AAAI Conference on Weblogs and Social Media (ICWSM)*. Ann Arbor, MI.

[8] McKinney, W. (2010) 'Data Structures for Statistical Computing in Python', in *Proceedings of the 9th Python in Science Conference*. Austin, TX, pp. 51–56.

[9] Pang, B. and Lee, L. (2008) 'Opinion mining and sentiment analysis', *Foundations and Trends in Information Retrieval*, 2(1–2), pp. 1–135.

[10] Spoonacular (2026) *Food API Documentation*. Available at: https://spoonacular.com/food-api/docs (Accessed: 27 March 2026).

[11] Kelleher, J. D. and Tierney, B. (2018) *Ciencia de datos*. MIT Press / Conocimientos Esenciales.

[12] Zhang, Y., Ai, Q., Chen, X. and Croft, W. B. (2018) 'Joint representation learning for top-n recommendation with heterogeneous information sources', in *Proceedings of the ACM International Conference on Web Search and Data Mining*.

Appendix A

Course Deliverable Traceability

A.1 Workshop and Final Artifact Files

- Final poster
- Final paper
- Workshop 1: data analysis and source validation.
- Workshop 2: technical architecture and Bronze/Silver pipeline definition.
- Workshop 3: data quality profiling and NLP refinement.
- Workshop 4: Gold layer, dashboards, integration fixes, and final validation.
- Architecture and README documents: Airflow/API architecture, web architecture, mobile architecture, installation instructions, and implementation README files.

A.2 Supplementary Evidence

The appendices summarize supporting evidence rather than reproducing raw datasets. The main evidence includes raw Bronze JSON files, Silver Parquet outputs, Gold governance and storytelling Parquet datasets, dashboard screenshots or running interfaces, and workshop reports documenting the iterative development process.

Appendix B

Architecture Diagrams

This appendix converts the project architecture diagrams into LaTeX-friendly tables and flow summaries. The original design uses Mermaid notation; the versions below preserve the same technical content without requiring external diagram rendering during compilation.

B.1 Docker Infrastructure

Table B.1: Docker Compose services and mounted resources.

Component	Responsibility	Dependencies and resources
airflow-init	One-shot setup that initializes the Airflow metadata schema.	Runs before the webserver and scheduler; writes to PostgreSQL.
airflow-webserver	Exposes the Airflow UI at `localhost:8080` using the default `admin/admin` credentials.	Depends on a healthy initialization step and connects to PostgreSQL.
airflow-scheduler	Executes DAG tasks according to the configured schedule.	Reads DAG files and accesses Bronze, Silver, Gold, and environment volumes.
PostgreSQL	Stores Airflow metadata.	Exposes the metadata database on port 5432.
Custom Airflow image	Provides the runtime required by the pipeline.	Built from `apache/airflow:2.10.2-python3.11` with Java JDK, PySpark 3.5.1, pandas, NLTK, spaCy, Playwright, Firefox, and related dependencies.

The mounted host resources are `./airflow/dags/`, `./datalake_bronze/`, `./datalake_silver/`, `./datalake_gold/`, and `.env`. These map into the Airflow container so that DAG definitions, data-lake layers, and the `SPOONACULAR_API_KEY` are available at runtime.

B.2 Unified Airflow DAG

The DAG flow is: `t1 -> t2 -> t3 -> t4 -> t5 -> t6 -> t7 -> t8 -> t9 -> t10`. Its schedule is configured as manual execution with `schedule_interval=None`,

Table B.2: Sequential tasks in `nyc_gastronomy_pipeline`.

Step	Task	Script or function	Output
t1	extract_eater_ny	`WebScrapping_NY.py` `main()`	Bronze Eater NY JSON files.
t2	clean_eater_ny	`preprocess_datasets.py` `preprocess_web_only()`	Deduplicated Silver web Parquet files.
t3	extract_opentable	`opentable_ingestion.py` `main()`	Bronze OpenTable review JSON files.
t4	clean_opentable	`preprocess_datasets.py` `preprocess_opentable_only()`	Deduplicated Silver review Parquet files.
t5	analyze_trends	`trend_analysis.py` `extract_trending_keywords()`	XCom keyword list from articles and reviews.
t6	search_ spoonacular	`api_ingestion.py` `search_by_keywords()`	Bronze Spoonacular JSON files.
t7	clean_recipes	`preprocess_datasets.py` `preprocess_api_only()`	Silver recipe Parquet files.
t8	prepare_gold	`spark_transform.py` `spark_governance.py` `spark_storytelling.py`	Gold recipes, articles, reviews, governance, and storytelling Parquet outputs.
t9	upload_s3	Configured export helper	Optional latest Gold upload when cloud storage is configured.
t10	cleanup	Retention helper	Removal of old local partitions according to the retention window.

`max_active_runs` is set to `1` to avoid concurrent Spark JVMs, and each task has one retry with a five-minute delay.

B.3 Web Scraping and API Integration

The Spoonacular client also supports an ingredient mode using `includeIngredients`, but the unified pipeline uses keyword mode so that external recipe retrieval is driven by the current NLP trend signal.

B.4 Silver NLP Pipeline

After cleaning, a TF-IDF vectorizer with `max_features=500` identifies distinctive terms in the current article and review batch. The `is_food_token()` filter then combines a curated cuisine vocabulary with spaCy cosine similarity against a food anchor vector using a threshold of `0.35`. Review-oriented stopwords such as service, staff, dinner, amazing, and wonderful are removed so that generic opinion words do not drive recipe searches. The resulting gastronomic keywords are returned through XCom for the Spoonacular task.

B.5 Gold Spark Processing

Each Gold script creates and destroys its own `SparkSession`; this keeps the execution sequential and avoids running multiple Spark JVMs at the same time inside the local Airflow

Table B.3: External ingestion sequence summaries.

Flow	Main sequence	Persisted fields or files
Eater NY scraping	Airflow calls `main()`, requests `ny.eater.com/rss/index.xml`, detects Atom or RSS format, parses up to 20 articles, removes photo attribution, checks existing `article_id` values, and writes only new records.	JSON under `bronze/webscraping/YYYY-MM-DD/` with source, title, URL, article ID, author, summary, categories, published date, and extraction timestamp.
OpenTable reviews	Airflow launches Playwright with Firefox, opens OpenTable, extracts browser session cookies, calls the GraphQL review search operation, paginates restaurant reviews, and falls back to a curated NYC restaurant list when discovery fails.	JSON under `bronze/opentable/YYYY-MM-DD/` with review ID, review text, overall rating, food rating, service rating, ambience rating, dining date, restaurant slug, and user metro.
Spoonacular API	Trend analysis returns keywords through XCom; API ingestion calls `SpoonacularClient.search(keyword, mode="keyword")` and queries `/recipes/complexSearch` with recipe information enabled and five results per keyword.	JSON under `bronze/api/YYYY-MM-DD/` with recipe ID, title, summary HTML, scores, dietary flags, cuisines, dish types, and diets.

Table B.4: Ordered transformations applied by `clean_nlp_text()`.

No.	Transformation	Purpose
1	Remove photo attribution	Removes text before the first encoded separator within the first 300 characters.
2	HTML unescape	Converts encoded HTML entities into readable characters.
3	URL removal	Deletes `https://` and `www.` links.
4	Lowercasing	Normalizes lexical variants such as `Best Pizza` and `best pizza`.
5	Punctuation removal	Replaces non-word and non-space characters with spaces.
6	Whitespace normalization	Collapses repeated spaces into a single separator.
7	Token filtering	Removes NLTK stopwords, domain stopwords, one-character tokens, and numeric-only tokens.

Table B.5: Gold-layer Spark jobs executed by `prepare_gold`.

Spark script	Processing logic	Gold output

`spark_transform.py`	Reads Silver API, web, and review data, casts conflicting fields such as `license`, unions schemas with missing columns allowed, selects dashboard columns, derives `dietary_tags`, deduplicates by ID, and writes coalesced Parquet.	`gold_recipes_*.parquet`
		`gold_articles_*.parquet`
		`gold_reviews_*.parquet`

`spark_governance.py`	Computes null rate, record count, source-file count, duplicate rate, schema compliance, outlier rate, NLP compression ratio, and ingestion date coverage.	`governance_*.parquet` long KPI format
`spark_storytelling.py`	Applies a VADER sentiment UDF and computes dashboard aggregations, including article sentiment, review sentiment, combined sentiment, monthly trends, top keywords, keyword sentiment, reviews by restaurant, recipe-trend alignment, and dietary breakdown.	`storytelling_*.parquet` long analytical format

environment.

B.6 Medallion Data Flow

Table B.6: Medallion architecture from raw ingestion to dashboard-ready outputs.

Layer	Location	Transformation	Consumer
Bronze	`bronze/webscraping/`, `bronze/opentable/`, and `bronze/api/`	Stores raw JSON from Eater NY, OpenTable, and Spoonacular without analytical transformation.	Silver preprocessing jobs.
Silver	`silver/webscraping/`, `silver/opentable/`, and `silver/api/`	Deduplicates articles and reviews, cleans NLP text, converts recipe arrays to strings, strips HTML summaries, and normalizes numeric types.	TF-IDF trend extraction and Gold Spark jobs.
Gold	`gold/`	Produces curated recipe, article, review, governance, and storytelling Parquet datasets with one coalesced output file per product.	Sentiment and gastronomy dashboards.

The end-to-end path is: Eater NY feed and OpenTable reviews to Bronze JSON, Silver article and review Parquet, combined trend keywords, Spoonacular API, Bronze API JSON, Silver API Parquet, Gold Spark products, optional S3 export, retention cleanup, and finally the dashboard layer for sentiment, trends, keywords, review context, recipe alignment, and dietary analysis.

B.7 Master Architecture Summary

Table B.7: End-to-end system context.

Area	Role in the architecture
External sources	Eater NY provides the Atom/RSS culinary media feed; OpenTable provides diner reviews and ratings through the implemented browser-assisted extraction; Spoonacular provides structured recipe metadata through its REST API.
Docker infrastructure	PostgreSQL, `airflow-init`, `airflow-webserver`, and `airflow-scheduler` run the local orchestration environment.
Airflow runtime	Apache Airflow 2.10.2, Python 3.11, PySpark 3.5.1, Java, Playwright, and Firefox execute the manually triggered unified DAG.
Data lake	Docker-mounted Bronze, Silver, and Gold directories persist JSON and Parquet assets across pipeline runs.
Consumers	The dashboard consumes Gold recipes, Gold articles, Gold reviews, and storytelling aggregations to expose sentiment, trend, keyword, source, review, recipe-alignment, and dietary views.

Table B.8: Diagram types represented in this appendix.

Architecture view	Diagram type	Purpose
Docker infrastructure	Component / deployment	Shows services, dependencies, volumes, and runtime image.
Airflow DAG	Activity / flowchart	Shows task order and XCom keyword exchange.
Web scraping	Sequence	Shows Eater NY feed scraping, OpenTable browser-assisted review extraction, deduplication, and Bronze persistence.
Spoonacular API	Sequence	Shows NLP-driven API retrieval and Bronze recipe persistence.
NLP pipeline	Activity	Shows text-cleaning and food-keyword extraction stages.
Gold Spark	Component	Shows scripts, inputs, metrics, aggregations, and outputs.
Medallion flow	Data flow	Shows Bronze to Silver to Gold transformations.
Master architecture	System context	Shows the complete infrastructure, pipeline, data lake, and dashboard relationship.